Forecasting Weekly NFL Injury Reports

STAT 460/560 Final Report

Elijah Homan, Logan Worrell, Thomas Shields

2026-12-08

MAKE CHANGES

Executive summary

We asked whether the number of NFL injury reports per week has changed since 2016, and how many reports to expect in each week of the 2026 regular season. Apart from an unusually high 2016 season, weekly injury reports have held steady at about 140 to 150 per week, with no sign of a rise or a fall. The most reliable pattern is within each season: Week 1 has far fewer reports than any other week, and counts level off by about Week 3. We forecast about 110 reports in Week 1 of 2026 and about 128 to 164 per week for the rest of the season. Any single week can easily be 30 reports above or below its forecast. We recommend that team medical staffs plan for a lighter opening week and a steady workload afterward, and that the league treat week-to-week swings within this range as normal variation.

Introduction

Every NFL team must file injury reports during the regular season, listing each player who is limited or held out because of an injury. The number of these reports is a simple, public measure of how much injury the league's medical staffs are managing each week. The NFL and the NFL Players Association have spent years changing rules to make the game safer, so they have a direct interest in whether injury reports are rising or falling. Team medical and training staffs also benefit from knowing how busy each week of the season is likely to be.

This report forecasts the weekly number of injury reports for one full regular season (17 weeks) ahead. We chose this horizon because staffing and planning decisions are made season by season. We compared nine forecasting methods, chose the one that predicted held-out seasons most accurately, and used it to forecast the 2026 season.

Data

Source and cleaning

The data come from the nflverse project, an open-source collection of NFL data, and were downloaded with the `nflreadr` R package. Each record is one player listed on one team's injury report in one week. We kept the 2016 to 2025 regular seasons and counted the records in each week. We made three choices:

- We kept only Weeks 1 to 17. Since 2021 the regular season has had 18 weeks; dropping Week 18 keeps every season the same length.

- We removed records labeled “Not injury related - resting player”, which are rest days, not injuries.
- We removed the few records with no injury listed.

The result is 170 weekly counts (10 seasons of 17 weeks), with no missing weeks. Note that a count is the number of reports, not the number of new injuries: a player who misses three weeks with one injury appears on three reports.

The NFL regular season does not run all year, so the weeks are not evenly spaced on a calendar. We number the weeks consecutively (1 to 170), so that Week 1 of one season directly follows Week 17 of the previous season. One consequence is that our software cannot detect the 17-week season on its own, so we specify the 17-week seasonal period in every model. We did not transform the counts (for example, with logarithms), because their variability does not grow with their level.

What the data show

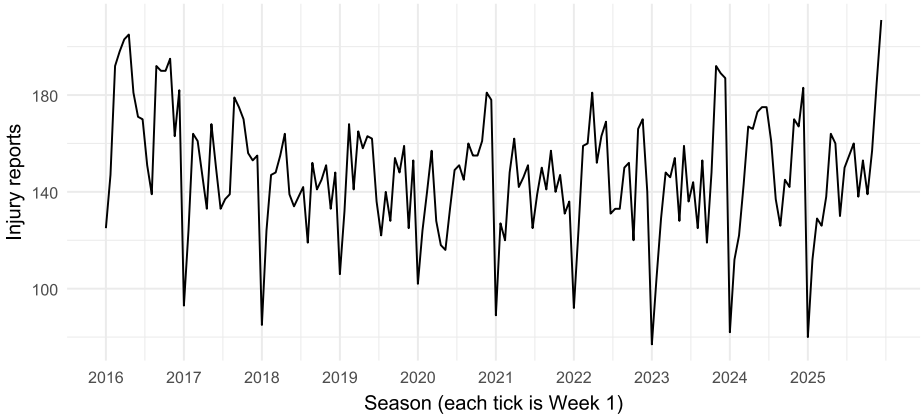


Figure 1: Weekly NFL injury reports, Weeks 1–17 of the 2016–2025 regular seasons.

Figure 1 shows the full series. Each season starts with a sharp dip in Week 1. The 2016 season is clearly higher than the rest, averaging 176 reports per week, compared with 138 to 150 in every season since. After 2016 there is no visible upward or downward trend.

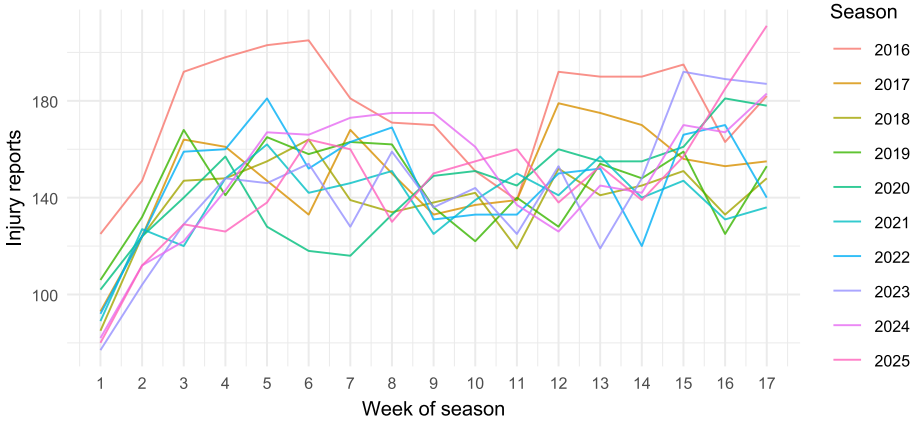


Figure 2: Injury reports by week of the season, one line per season.

Figure 2 overlays the seasons. Week 1 is the lowest week in every season, counts rise quickly through Week 3, and the remaining weeks fluctuate around a plateau. Beyond the Week 1 dip, the ups and downs differ from season to season.

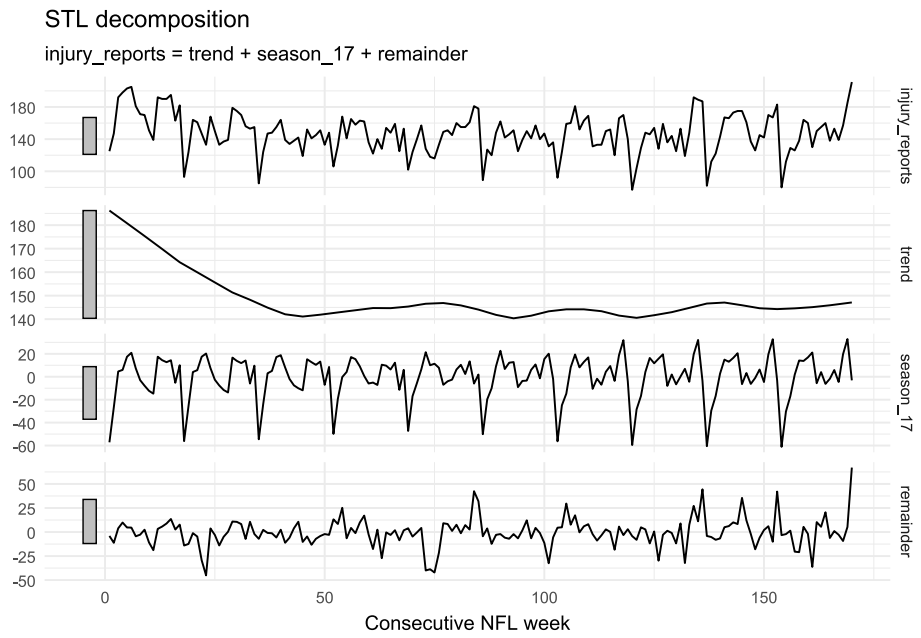


Figure 3: STL decomposition into trend, week-of-season pattern and remainder.

We split the series into a trend, a repeating week-of-season pattern, and a remainder using an STL decomposition (Figure 3). We used a seasonal window of 7 seasons, so the season shape can change slowly, and the default trend window, which is longer than one season. (In Milestone 2 we used a trend window of 7 weeks, which let the trend absorb part of the Week 1 dip.) The trend falls from 2016 to 2017 and is flat afterward. The seasonal strength is 0.68 on a scale from 0 to 1, so the week-of-season pattern explains much of the variation. The trend strength is 0.41, which comes almost entirely from the 2016 drop. The remainder is large, so individual weeks are noisy and hard to predict.

Revisions in response to Milestone 2 feedback: [to be added when feedback is received].

Methods

Models

We compared nine methods. The first four are simple benchmarks that any useful model should beat:

- Mean: every future week equals the average of all past weeks.
- Naive: every future week equals the most recent week.
- Seasonal naive: each week of next season equals the same week of last season.
- Drift: a straight line continuing from the first to the last observation.

The next three are the standard methods from our course:

- ETS (exponential smoothing): a weighted average of past values, with more weight on recent weeks, for the level, trend and week-of-season pattern. The automatically chosen form was ETS(M,Ad,A): multiplicative errors, a damped trend and an additive season.
- ARIMA: models each week from changes relative to recent weeks and to the same week in past seasons. The automatically chosen model was ARIMA(1,1,1)(2,1,0) with a 17-week season.
- Regression: a linear trend plus a separate average level for each week of the season.

In Milestone 3, seasonal naive beat all three. We suspected two reasons: the unusually high 2016 season pulls the other models upward, and those models use their extra flexibility to fit noise. To test this, we added two models:

- Regression + 2016: the regression above with an extra term (a dummy variable) that lets the 2016 season have its own level.
- Dynamic regression: the same week-of-season levels and 2016 term, but with errors that are allowed to carry over from one week to the next (regression with ARIMA errors).

Both new models need future values of their predictors, the week of the season and the 2016 indicator. These are known in advance (the indicator is 0 for 2026), so the forecasts are genuine ex-ante forecasts.

Evaluation

We evaluated every model in two ways. First, we trained on 2016 to 2024 and forecast the 2025 season, which none of the models had seen. Second, because a single season can be unusual, we used time series cross-validation. We trained on 2016 to 2020 and forecast 2021, then trained on 2016 to 2021 and forecast 2022, and so on, giving five forecast seasons. We chose the final model by cross-validation, since it averages over five seasons rather than one. We measured accuracy with four measures: RMSE and MAE (typical errors, in reports per week), MASE (errors relative to seasonal naive, where values below 1 beat it), and CRPS (which also scores how well the prediction intervals fit the outcome).

Final model

The final model is the dynamic regression. For week t,

$$y_t = \beta_0 + \sum_{j=2}^{17} \gamma_j \, w_{j,t} + \delta \, d_t + \eta_t, \qquad \eta_t = \phi \, \eta_{t-1} + \varepsilon_t, \qquad \varepsilon_t \sim N(0, \sigma^2),$$

where y_t is the number of injury reports, $w_{j,t}$ is 1 if week t is Week j of a season (Week 1 is the baseline), d_t is 1 during the 2016 season, and η_t is an error that carries over from week to week. Re-fit on all ten seasons, the 2016 season is 30 reports per week higher than other seasons (δ), and $\phi = 0.4$, so about 40% of an unusually high or low week carries into the next week. The model has no trend term. A trend added to the regression model was small and not statistically significant, which is consistent with injury reports being flat since 2017.

Results

Accuracy comparison

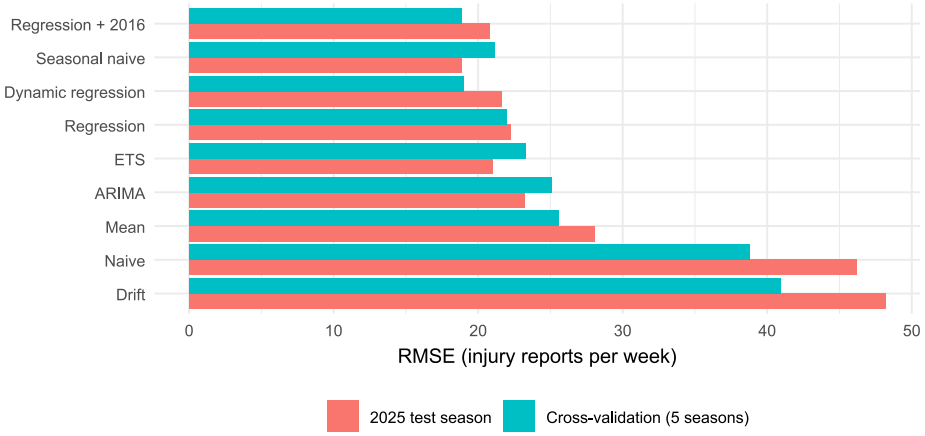


Figure 4: Forecast error (RMSE) of each method on the 2025 test season and in cross-validation. Lower is better.

```
# A tibble: 9 × 5
  Model               RMSE   MAE  MASE  CRPS
  <chr>              <dbl> <dbl> <dbl> <dbl>
1 Regression + 2016   18.8  15.1  0.8   10.7
2 Dynamic regression  19.0  15.3  0.81  10.9
3 Seasonal naive      21.1  17.2  0.92  12.0
4 Regression          21.9  17.3  0.92  12.4
5 ETS                 23.3  19.5  1.04  13.4
6 ARIMA               25.1  20.6  1.1   14.2
7 Mean                25.6  19.5  1.04  14.1
8 Naive               38.8  32.0  1.71  26.7
9 Drift               40.9  34.3  1.83  28.2
```

Figure 4 and the table above (cross-validation results) summarize the comparison. The two new models are the most accurate in cross-validation: the regression with a 2016 term has an RMSE of 18.8 and the dynamic regression 19, compared with 21.1 for seasonal naive. They also have the lowest MASE and CRPS. ETS and ARIMA do not beat seasonal naive. This supports our explanation from Milestone 3: once the 2016 season is allowed its own level, a model that averages the week-of-season pattern over many seasons forecasts better than simply repeating last season.

On the 2025 test season alone, seasonal naive is still the most accurate (RMSE 18.9, against 21.6 for the dynamic regression), because 2025 happened to look very similar to 2024. We give more weight to the five cross-validation seasons than to this one season. The regression with a 2016 term and the dynamic regression are nearly tied in cross-validation. We chose the dynamic regression because its residuals are much closer to random noise than those of either competitor (see below), so its prediction intervals are more trustworthy.

Residual diagnostics

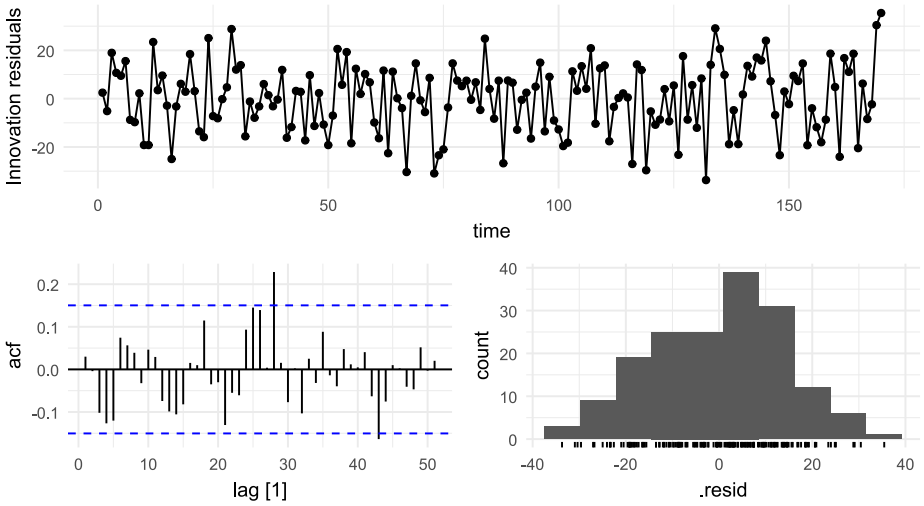


Figure 5: Residuals of the final model: over time, their autocorrelation (ACF), and their distribution.

A good forecasting model leaves residuals (the parts it cannot explain) that look like random noise. Figure 5 shows the final model's residuals. They are centered on zero, show no pattern over time, and have a roughly bell-shaped distribution, which supports the normal distribution used for the prediction intervals. Most autocorrelations fall inside the blue bounds, but two do not: a positive one at lag 28 and a negative one at lag 43. The Ljung–Box test, which checks the first 34 autocorrelations together, gives borderline evidence of remaining structure ($p = 0.04$). The two spikes do not fall at the seasonal lags (17 or 34), so we see no obvious way to model them, and they are small. For comparison, the same test strongly rejects for seasonal naive and for the regression with a 2016 term ($p < 0.001$ for both), whose residuals still contain clear week-to-week carry-over. The final model is therefore not perfect, but it is the best of the three, and its intervals should be close to correct.

Forecasts for the 2026 season

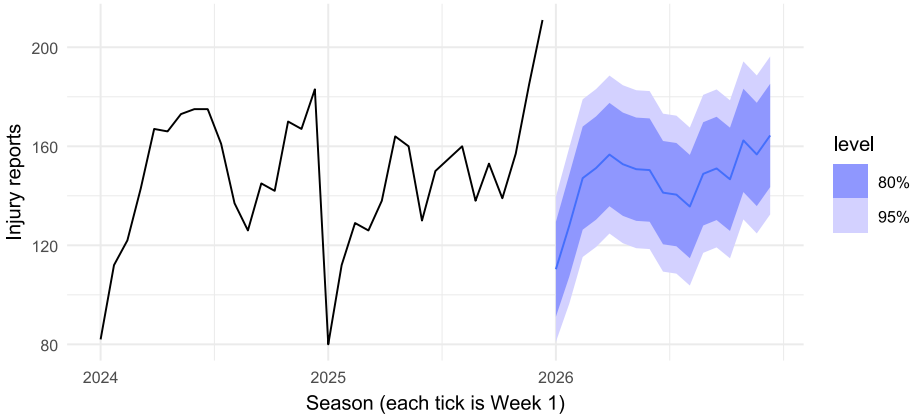


Figure 6: Forecast of weekly injury reports for the 2026 season, with 80% and 95% prediction intervals. The 2024 and 2025 seasons are shown for context.

```
# A tibble: 17 × 4
    Week Forecast `80% interval` `95% interval`
   <dbl>    <dbl> <chr>          <chr>
 1     1      110 91–129         81–140
 2     2      128 107–149        96–159
 3     3      147 126–168        115–179
 4     4      151 130–172        119–183
 5     5      157 136–178        125–189
 6     6      153 132–174        121–185
 7     7      151 130–172        119–183
 8     8      150 130–171        118–182
 9     9      141 120–162        109–173
10    10      140 120–161        109–172
11    11      136 115–157        104–168
12    12      149 128–170        117–181
13    13      151 130–172        119–183
14    14      147 126–168        115–179
15    15      162 142–183        130–194
16    16      157 136–178        125–189
17    17      164 144–185        132–196
```

Figure 6 and the table above give the 2026 forecasts from the final model, re-fit on all ten seasons. We expect about 110 injury reports in Week 1, then about 128 to 164 per week for the rest of the season. The 95% intervals are about plus or minus 32 reports wide: in 19 out of 20 weeks, the actual count should fall inside them. The Week 1 forecast is higher than recent Week 1 counts (about 80 in each of 2023 to 2025). The model's typical Week 1 level is 92 reports, and it adds about 18 because it carries part of the very high final week of 2025 (211 reports) into the next week.

Discussion

For the NFL and the NFL Players Association, the main finding is that the weekly volume of injury reports has not changed meaningfully since 2017. Whatever the effects of safety rule changes over this period, they have not shown up as a rise or fall in the number of reports. For team medical staffs, the forecasts support a simple planning rule: expect a much lighter Week 1, then a steady workload of roughly 128 to 164 league-wide reports per week. A single week well inside the intervals should not be read as a warning sign.

Several assumptions deserve caution.

- Reports are not injuries. A long injury produces many reports, so a season with a few long injuries could look the same as a season with many short ones. Our forecasts are about workload, not injury risk.
- The 2016 season. We do not know why 2016 was so much higher; it could reflect a change in how teams filled out reports. Our model treats it as a one-off. If a similar shift happened again, the forecasts would be too low until the model was updated.
- Carry-over across the off-season. Because our weeks are numbered consecutively, the model treats Week 17 and the following Week 1 as neighbors, even though they are months apart. This is why our

Week 1 forecast is pulled upward by the end of 2025. If that carry-over is not real, the Week 1 forecast is too high by about 18 reports.

- Remaining autocorrelation. The final model's residuals are borderline on the Ljung–Box test. If the leftover structure is real, the prediction intervals may be slightly too narrow.
- Schedule changes. The 2020 season was played under COVID-19 rules, and since 2021 the season has had 18 weeks, of which we use only the first 17. Week 18 may behave differently, and the extra game could change the pattern late in the season.
- Normal errors and stable conditions. The prediction intervals assume the errors are roughly normal and the future behaves like the past. A major rule or reporting change would make the intervals too narrow.

With more time or data, we would collect the number of distinct injuries and the games missed, which measure injury risk more directly than report counts. We would add Week 18 and test whether the 18-week schedule changed the late-season pattern. Finally, we would forecast individual teams or injury types, which would be more useful to a single team's staff than a league-wide total.

References

Hyndman, R. J., & Athanasopoulos, G. (2021). Forecasting: principles and practice (3rd ed.). OTexts. https://otexts.com/fpp3/

nflverse. (2025). nflverse-data: Automated NFL data repository. https://github.com/nflverse/nflverse-data

Ho, T., & Carl, S. (2025). nflreadr: Download 'nflverse' data (R package version 1.5.1). https://nflreadr.nflverse.com

O'Hara-Wild, M., Hyndman, R., & Wang, E. fable: Forecasting models for tidy time series (R package version 0.5.0).

O'Hara-Wild, M., Hyndman, R., & Wang, E. feasts: Feature extraction and statistics for time series (R package version 0.5.0).

Wang, E., Cook, D., & Hyndman, R. J. (2020). A new tidy data structure to support exploration and modeling of temporal data. Journal of Computational and Graphical Statistics, 29(3), 466–478.

Wickham, H., et al. (2019). Welcome to the tidyverse. Journal of Open Source Software, 4(43), 1686.

R Core Team. (2024). R: A language and environment for statistical computing (version 4.4.2). R Foundation for Statistical Computing.

Allaire, J. J., et al. Quarto (version 1.10). https://quarto.org

Appendix

Contribution statement

- Thomas Shields: data import and cleaning, train/test and cross-validation setup, ARIMA models and unit-root tests.
- Logan Worrell: benchmark and ETS models, regression models, accuracy comparison and residual diagnostics.
- Elijah Homan: writing and editing.

[Group: edit this list so that it describes what each member did across all milestones.]

Code

The complete code below reproduces every figure and table in this report.

```
library(nflreadr)
library(tidyverse)
library(fpp3)

theme_set(theme_minimal(base_size = 10))

# Load NFL injury reports from 2016–2025 (nflverse)
injuries <- load_injuries(2016:2025)

# Weekly counts of injury-report records, Weeks 1–17 of each regular season
weekly_injuries <- injuries |>
  filter(week %in% 1:17) |>
  filter(
    !is.na(report_primary_injury),
    report_primary_injury != "Not injury related - resting player"
  ) |>
  group_by(season, week) |>
  summarise(injury_reports = n(), .groups = "drop") |>
  arrange(season, week) |>
  mutate(
    time = (season - 2016) * 17 + week,   # consecutive NFL week, 1–170
    yr2016 = as.integer(season == 2016)   # level-shift dummy for the 2016 season
  ) |>
  as_tsibble(index = time)

# Training set: 2016–2024; test set: 2025
train <- weekly_injuries |> filter(time <= 153)

# Future predictor values: week of season, and 0 for the 2016 dummy
add_future_x <- function(future) {
  future |> mutate(week = (time - 1) %% 17 + 1, yr2016 = 0L)
}
```

```r
# All candidate models (the seasonal period of 17 weeks is always stated explicitly)
fit_models <- function(data) {
  data |>
    model(
      Mean = MEAN(injury_reports),
      Naive = NAIVE(injury_reports),
      `Seasonal naive` = SNAIVE(injury_reports ~ lag(17)),
      Drift = RW(injury_reports ~ drift()),
      ETS = ETS(injury_reports ~ season(period = 17)),
      ARIMA = ARIMA(injury_reports ~ PDQ(period = 17)),
      Regression = TSLM(injury_reports ~ trend() + season(period = 17)),
      `Regression + 2016` = TSLM(injury_reports ~ trend() + factor(week) + yr2016),
       `Dynamic regression` = ARIMA(injury_reports ~ factor(week) + yr2016 + PDQ(0,
0, 0))
    )
}

acc_measures <- list(RMSE = RMSE, MAE = MAE, MASE = MASE, CRPS = CRPS)

# Season averages used in the text
season_means <- weekly_injuries |>
  as_tibble() |>
  group_by(season) |>
  summarise(mean_reports = mean(injury_reports))
# Test-set evaluation: train on 2016–2024, forecast the 17 weeks of 2025
fit_train <- fit_models(train)
fc_test <- fit_train |> forecast(new_data = add_future_x(new_data(train, 17)))
acc_test <- accuracy(fc_test, weekly_injuries, measures = acc_measures, .period = 17)

# Time series cross-validation: 5 expanding folds, each forecasting one full season
cv_data <- weekly_injuries |>
  stretch_tsibble(.init = 85, .step = 17) |>
  filter(.id < max(.id))
fc_cv <- fit_models(cv_data) |>
  forecast(new_data = add_future_x(new_data(cv_data, 17)))
acc_cv <- accuracy(fc_cv, weekly_injuries, measures = acc_measures, .period = 17)

acc_all <- bind_rows(
  acc_test |> mutate(evaluation = "2025 test season"),
  acc_cv |> mutate(evaluation = "Cross-validation (5 seasons)")
)

# Helper to pull a value from the accuracy results for the text
acc_value <- function(m, measure, eval) {
  acc_all |>
    filter(.model == m, evaluation == eval) |>
```

```
    pull({{ measure }}) |>
    round(1)
}
cv_name <- "Cross-validation (5 seasons)"
test_name <- "2025 test season"
# Final model: dynamic regression, re-fit on all ten seasons
fit_final <- weekly_injuries |>
  model(dynreg = ARIMA(injury_reports ~ factor(week) + yr2016 + PDQ(0, 0, 0)))

fc_2026 <- fit_final |>
  forecast(new_data = add_future_x(new_data(weekly_injuries, 17)))

fc_2026_table <- fc_2026 |>
  hilo(level = c(80, 95)) |>
  as_tibble() |>
  transmute(
    Week = time - 170,
    Forecast = round(.mean),
    `80% interval` = paste0(round(`80%`$lower), "–", round(`80%`$upper)),
    `95% interval` = paste0(round(`95%`$lower), "–", round(`95%`$upper))
  )

final_coefs <- tidy(fit_final)
phi <- final_coefs |> filter(term == "ar1") |> pull(estimate)
delta_2016 <- final_coefs |> filter(term == "yr2016") |> pull(estimate)
week1_fc <- fc_2026_table$Forecast[1]
rest_range <- range(fc_2026_table$Forecast[-1])
half_width_95 <- fc_2026 |>
  hilo(level = 95) |>
  mutate(hw = (`95%`$upper - `95%`$lower) / 2) |>
  pull(hw) |>
  mean() |>
  round()

# Week 1 level without carry-over (intercept), to measure the carry-over into Week 1
of 2026
week1_base <- final_coefs |> filter(term == "intercept") |> pull(estimate)
week1_carry <- round(week1_fc - week1_base)

# Residual diagnostics for the final model (Ljung–Box, lag = 2 x 17, one AR coefficient)
lb_final <- fit_final |>
  augment() |>
  features(.innov, ljung_box, lag = 34, dof = 1)

# For comparison: the two closest competitors, re-fit on all ten seasons
lb_others <- weekly_injuries |>
```

```
  model(
    `Seasonal naive` = SNAIVE(injury_reports ~ lag(17)),
    `Regression + 2016` = TSLM(injury_reports ~ trend() + factor(week) + yr2016)
  ) |>
  augment() |>
  features(.innov, ljung_box, lag = 34)
weekly_injuries |>
  autoplot(injury_reports) +
  scale_x_continuous(breaks = (0:9) * 17 + 1, labels = 2016:2025) +
  labs(x = "Season (each tick is Week 1)", y = "Injury reports")
weekly_injuries |>
  as_tibble() |>
  ggplot(aes(x = week, y = injury_reports, color = factor(season), group = season)) +
  geom_line(alpha = 0.8) +
  scale_x_continuous(breaks = 1:17) +
  labs(x = "Week of season", y = "Injury reports", color = "Season")
weekly_injuries |>
  model(STL(injury_reports ~ season(period = 17, window = 7), robust = TRUE)) |>
  components() |>
  autoplot() +
  labs(x = "Consecutive NFL week")
stl_features <- weekly_injuries |> features(injury_reports, feat_stl, .period = 17)
acc_all |>
  ggplot(aes(x = RMSE, y = reorder(.model, -RMSE), fill = evaluation)) +
  geom_col(position = "dodge") +
  labs(x = "RMSE (injury reports per week)", y = NULL, fill = NULL) +
  theme(legend.position = "bottom")
acc_cv |>
  select(Model = .model, RMSE, MAE, MASE, CRPS) |>
  arrange(RMSE) |>
  mutate(across(where(is.numeric), \(x) round(x, 2))) |>
  print(width = Inf)
fit_final |>
  gg_tsresiduals(lag_max = 51)
fc_2026 |>
  autoplot(weekly_injuries |> filter(time > 136), level = c(80, 95)) +
  scale_x_continuous(breaks = (8:10) * 17 + 1, labels = 2024:2026) +
  labs(x = "Season (each tick is Week 1)", y = "Injury reports", level = "Interval")
print(fc_2026_table, n = 17)
```